Mentor: Darsi Gangothri

Session 6 - PyTorch & Transformers

Date: Sunday, 29 Jun 2026

Module: 2.2 (Part 1) - ML & Deep Learning with PyTorch

Course: LLM Engineering Masterclass

What We Covered

Session Overview

This session begins Module 2.2 by covering PyTorch fundamentals, writing a training loop from scratch, and understanding the full Transformer architecture. Session 5 introduced neural networks and self-attention conceptually. Today we code it and go deeper.

By the end of this session, learners will understand: - What PyTorch is and why it matters for LLMs - What tensors are and how they differ from Python lists - How to build a neural network with nn.Module - How to write a training loop and watch a model learn - What multi-head attention, positional encoding, and decoder-only architecture are - Why every LLM uses the Transformer architecture

---

Topic 1: PyTorch Fundamentals

What is PyTorch?

PyTorch is a Python library for building and training neural networks. Every major LLM (Llama, Mistral, GPT, Gemini) was built with PyTorch. It is the industry standard toolkit.

Tensors

A tensor is a container for numbers. That is it. A structured container that holds numbers.

- A single number (5) is a 0D tensor (scalar)
- A list of numbers ([1, 2, 3]) is a 1D tensor (vector)

- A table of numbers (rows and columns) is a 2D tensor (matrix)
- You can keep going: 3D, 4D, etc.

Tensors look like Python lists but have two superpowers:

1. **GPU** -- tensors can run on a GPU for massive parallel math. Python lists cannot.
2. **Autograd** -- PyTorch automatically tracks operations and computes gradients (how much each weight should change). This is what makes training possible.

Creating Tensors

```
import torch

numbers = torch.tensor([1, 2, 3, 4, 5])
print(numbers)        # tensor([1, 2, 3, 4, 5])
print(numbers.shape)  # torch.Size([5])

table = torch.tensor([[1, 2, 3],
                      [4, 5, 6]])
print(table.shape)    # torch.Size([2, 3])
```

Useful shortcuts:

```
zeros = torch.zeros(3, 4)   # all zeros
ones = torch.ones(2, 2)     # all ones
random = torch.randn(2, 3)  # random numbers (normal distribution)
```

The `0.` notation (trailing dot) in tensor output means floating-point (decimal). `0.` is the same as `0.0`. PyTorch defaults to floats because neural network math needs decimals.

Basic Math

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

print(a + b)      # tensor([5., 7., 9.])
print(a * b)      # tensor([ 4., 10., 18.])
print(a.mean())   # tensor(2.)
```

nn.Module -- Building a Neural Network

Every neural network in PyTorch is built using `nn.Module`. It has two parts:

- `__init__` -- define what layers the network has

- `forward` -- define how data flows through those layers

```
import torch.nn as nn

class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(1, 1)

    def forward(self, x):
        return self.layer(x)
```

`nn.Linear(1, 1)` is a single layer that does: output = input x weight + bias. One number in, one number out.

`super().__init__()` calls the parent class setup -- required for nn.Module to work.

When you print the model, it shows `bias=True`. The bias is the extra number added after multiplication (like the +3 in y = 2x + 3). It lets the model shift the output up or down.

Generators and named_parameters()

`model.named_parameters()` returns a generator -- a Python concept that gives items one at a time instead of all at once. Like a ticket counter: you get one ticket, then the next person goes.

PyTorch uses generators because real models have billions of parameters. Loading all into memory as a list would be wasteful.

Use a `for` loop to pull items from the generator:

```
for name, param in model.named_parameters():
    print(f"{name}: {param.data}")
```

Topic 2: Training Loop

The Concept

The training loop is the heart of deep learning. It repeats four steps:

1. **Forward pass** -- data goes in, prediction comes out
2. **Loss** -- compare prediction to correct answer (how far off?)
3. **Backward pass** -- trace back, figure out which weights caused the error

4. **Update** -- adjust those weights slightly

Analogy: archery practice. Shoot, see where it lands (loss), figure out what you did wrong (backward), adjust your stance (update), shoot again. After 500 arrows, you hit near the bullseye.

Setting Up the Problem

We teach the model y = 2x. It does not know the rule. It starts with random weights.

```
x_train = torch.tensor([[1.0], [2.0], [3.0], [4.0], [5.0]])
y_train = torch.tensor([[2.0], [4.0], [6.0], [8.0], [10.0]])
```

Training Components

```
model = SimpleModel()
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

- **MSELoss** -- Mean Squared Error. Measures how wrong predictions are. Squares the error so big mistakes are penalized more.
- **SGD** -- Stochastic Gradient Descent. Looks at gradients and takes a small step to reduce error.
- **lr=0.01** -- learning rate. How big each adjustment is. Too high = overshoot. Too low = learn very slowly.

The Loop

```
for epoch in range(500):
    predictions = model(x_train)
    loss = loss_fn(predictions, y_train)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    if (epoch + 1) % 50 == 0:
        print(f"Epoch {epoch+1:3d}, Loss: {loss.item():.4f}")
```

- `loss.backward()` -- computes gradients. A gradient tells us: "if I increase this weight slightly, how much does the loss change?"
- `optimizer.step()` -- adjusts weights using gradients
- `optimizer.zero_grad()` -- clears old gradients (PyTorch accumulates by default)

- `loss.item()` -- converts tensor to plain Python number
- `% 50` -- modulo operator (remainder). Prints every 50th epoch.
- `:.4f` -- format to 4 decimal places

Testing the Trained Model

```
test_input = torch.tensor([[6.0]])
prediction = model(test_input)
print(f"Input: 6.0, Predicted: {prediction.item():.2f}, Expected: 12.0")
```

The model predicts close to 12.0 for input 6 -- it learned y = 2x from just 5 examples, without being told the rule.

Key Insight

The model IS the weights. When you download a 4GB model file, that file is just billions of carefully tuned numbers. Training adjusts those numbers. That is all training is.

Topic 3: Full Transformer Architecture

Self-Attention Recap

For every word, the model asks: "Which other words should I pay attention to?"

"The cat sat on the mat because it was tired" -- when processing "it", the model pays strong attention to "cat" (what "it" refers to) and weak attention to other words.

Multi-Head Attention

One attention head looks from ONE angle. But language is complex. Multi-head attention runs multiple attention operations in parallel, each from a different perspective.

Analogy: three people reading the same email -- a grammar teacher checks structure, a domain expert checks meaning, a context analyst checks connections. Then they combine their notes.

Example sentence: "The bank approved the loan after reviewing the application."

- Head 1 (grammar): "approved" connects to "bank" (subject-verb)
- Head 2 (meaning): "loan" and "bank" are related finance concepts

- Head 3 (context): "reviewing" and "application" are the condition

Real models use 32+ attention heads.

Positional Encoding

The model receives all words at once (not one by one). Without position information, "dog bites man" and "man bites dog" look identical -- same words, same embeddings.

Positional encoding adds a position signal to each word's representation. Like seat numbers in a theater -- without them, you would not know the order.

Feed-Forward Network

After attention combines information from related words, a small neural network transforms each word's representation further. Think of it as "digesting" the attention output.

Residual Connections

After each layer (attention or feed-forward), the original input is added back. This is a safety net -- the original signal is never lost, even through many layers.

Layer Normalization

After each sub-layer, the numbers are normalized (scaled to a standard range). This keeps training stable and prevents numbers from growing too large or small.

Stacking Layers

One Transformer block = attention + feed-forward. Real models stack many blocks:

- GPT-2: 12 blocks
- Llama 3 (8B): 32 blocks
- Llama 3 (70B): 80 blocks

Early blocks learn simple patterns (grammar). Later blocks learn complex patterns (meaning, reasoning).

Encoder-Decoder vs Decoder-Only

Encoder-decoder -- used for translation. The encoder fully understands the input (English), then the decoder generates the output (French). Two separate components.

Decoder-only -- used for chatbots and text generation. It predicts the next token based on everything before it. No separate encoder needed because the input and output are the same conversation.

Every LLM you will use (ChatGPT, Llama, Gemini, Mistral) is decoder-only.

Session Summary

#	Topic	Key Takeaway
1	PyTorch Tensors	Containers for numbers with GPU + autograd superpowers
2	nn.Module	Blueprint: `__init__` (layers) + `forward` (data flow)
3	Training Loop	Forward -> Loss -> Backward -> Update. Loss goes down. Model learns.
4	Multi-Head Attention	Multiple perspectives looking at the same sentence simultaneously
5	Positional Encoding	How the model knows word ORDER (seat numbers)
6	Decoder-Only	Predicts next token based on everything before it. Every LLM uses this.

Homework

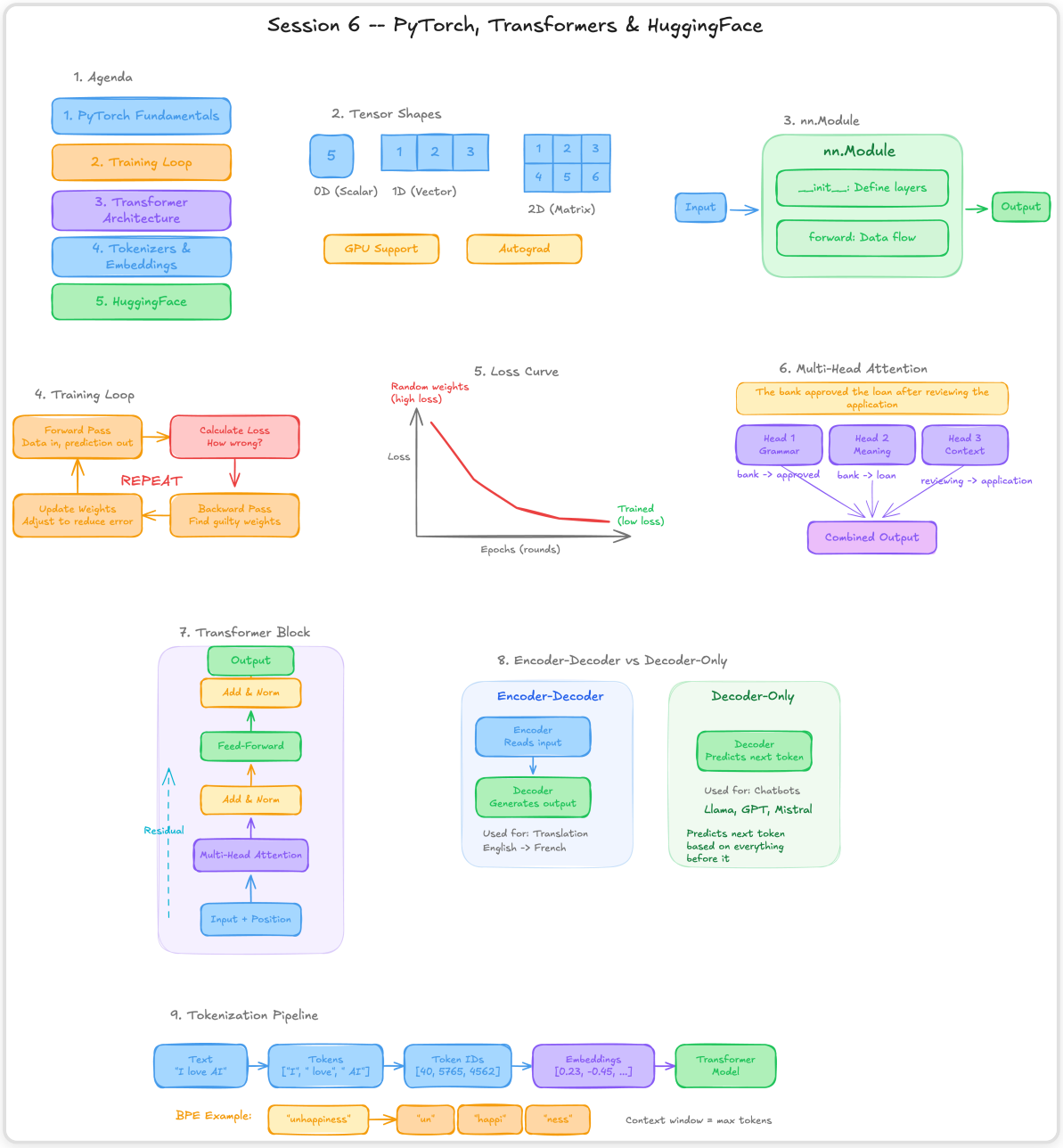
1. Run the training loop notebook. Change the learning rate to 0.1 and to 0.001. What happens to the loss? Compare.

Prep for Session 7

- Google Colab is sufficient (nothing new to install locally)
- Topics: Tokenizers and embeddings, HuggingFace Transformers library, open-source LLM landscape, quantization, Ollama deep dive

---

Session Whiteboard



Notebook Code (Full Reference)

All code from the session. Uses PyTorch (pre-installed on Colab). Setup: `pip install torch`

```
import torch
import torch.nn as nn

print(f"PyTorch version: {torch.__version__}")
```

```
# 1D tensor (a list of numbers)
numbers = torch.tensor([1, 2, 3, 4, 5])
print("1D tensor:", numbers)
print("Shape:", numbers.shape)
print("Type:", type(numbers))
```

```
# 2D tensor (a table -- rows and columns)
table = torch.tensor([[1, 2, 3],
                      [4, 5, 6]])
print("2D tensor:\n", table)
print("Shape:", table.shape)
```

```
# Useful tensor creation shortcuts
zeros = torch.zeros(3, 4)   # 3 rows, 4 columns, all zeros
ones = torch.ones(2, 2)     # 2 rows, 2 columns, all ones
random = torch.randn(2, 3)  # 2 rows, 3 columns, random numbers

print("Zeros:\n", zeros)
print("\nOnes:\n", ones)
print("\nRandom:\n", random)

# Notice the 0. and 1. -- the dot means floating-point (decimal)
# PyTorch defaults to floats because neural network math needs decimals
```

```
# Basic math on tensors
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

print("Add:", a + b)
print("Multiply:", a * b)
print("Mean:", a.mean())
```

```
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(1, 1)

    def forward(self, x):
        return self.layer(x)
```

```
# Create the model
model = SimpleModel()
print(model)

# named_parameters() returns a generator -- gives items one at a time
# Use a for loop to see the actual weights
print("\nModel weights (random starting values):")
for name, param in model.named_parameters():
    print(f"  {name}: {param.data}")
```

```
# The problem: teach the model that y = 2x
# Training data -- input and correct output pairs
x_train = torch.tensor([[1.0], [2.0], [3.0], [4.0], [5.0]])
y_train = torch.tensor([[2.0], [4.0], [6.0], [8.0], [10.0]])

print("Training data:")
for i in range(len(x_train)):
    print(f"  x={x_train[i].item():.0f} -> y={y_train[i].item():.0f}")
```

```
# Three things needed for training:
model = SimpleModel()                                  # Fresh model
(random weights)
loss_fn = nn.MSELoss()                                 # How to measure
error
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)  # How to adjust
weights

# lr = learning rate (how big each adjustment is)
```

```
# The Training Loop
for epoch in range(500):
    # 1. Forward pass
    predictions = model(x_train)

    # 2. Calculate loss
    loss = loss_fn(predictions, y_train)

    # 3. Backward pass
    loss.backward()

    # 4. Update weights
    optimizer.step()

    # 5. Clear gradients
    optimizer.zero_grad()

    # Print every 50 rounds
    if (epoch + 1) % 50 == 0:
        print(f"Epoch {epoch+1:3d}, Loss: {loss.item():.4f}")
```

```
# Test: what does the model predict for x = 6?
# (it never saw 6 during training!)
test_input = torch.tensor([[6.0]])
prediction = model(test_input)
print(f"Input: 6.0, Predicted: {prediction.item():.2f}, Expected: 12.0")

test_input_2 = torch.tensor([[10.0]])
prediction_2 = model(test_input_2)
print(f"Input: 10.0, Predicted: {prediction_2.item():.2f}, Expected: 20.0")
```

```
# What did the model learn?
print("Learned weights:")
for name, param in model.named_parameters():
    print(f"  {name}: {param.data}")

# Weight should be close to 2.0, bias close to 0.0
# Because the rule is y = 2x + 0
```

```
# Try lr=0.1 (too high?)
model_fast = SimpleModel()
optimizer_fast = torch.optim.SGD(model_fast.parameters(), lr=0.1)

print("--- Learning Rate = 0.1 ---")
for epoch in range(500):
    pred = model_fast(x_train)
    loss = loss_fn(pred, y_train)
    loss.backward()
    optimizer_fast.step()
    optimizer_fast.zero_grad()
    if (epoch + 1) % 100 == 0:
        print(f"Epoch {epoch+1:3d}, Loss: {loss.item():.6f}")

# Try lr=0.001 (too low?)
model_slow = SimpleModel()
optimizer_slow = torch.optim.SGD(model_slow.parameters(), lr=0.001)

print("\n--- Learning Rate = 0.001 ---")
for epoch in range(500):
    pred = model_slow(x_train)
    loss = loss_fn(pred, y_train)
    loss.backward()
    optimizer_slow.step()
    optimizer_slow.zero_grad()
    if (epoch + 1) % 100 == 0:
        print(f"Epoch {epoch+1:3d}, Loss: {loss.item():.6f}")
```